ChronoShield: Early Cyber Attack Detection via Temporal Attack Graphs & Risk Decay

DBMS Review 1 Report: Problem Definition, Requirement Analysis & NoSQL Database Design

Course: Database Management Systems (NoSQL Architecture)
Review Stage: Review 1 (Marks: 10)
Selected NoSQL Databases: MongoDB (Document Store) & Neo4j (Graph Database)
Target Domain: Cyber Security Threat Detection & Temporal Graph Analytics

1. Problem Statement, Objectives & Scope (2 Marks)

1.1 Background & Problem Statement

Modern cyber attacks executed by Advanced Persistent Threats (APTs) rarely consist of a single isolated malicious action. Instead, adversaries conduct low-and-slow tactics spanning days or weeks—such as initial credential exploitation, stealthy process injection, internal reconnaissance, lateral movement, and privilege escalation—to reach critical database assets. Traditional Intrusion Detection Systems (IDS) and relational Security Information and Event Management (SIEM) solutions fail because:

- **Stateless Log Processing:** They evaluate security logs in isolation or within small static rulesets, missing temporal multi-hop attack paths.
- **Relational Join Bottlenecks:** Querying N-hop connected entity paths in Relational DBMS produces exponential join computational complexity O(k^d), severely degrading query performance.
- **Lack of Explainable Causal Paths:** Traditional tools issue disjointed alerts without displaying the end-to-end attack graph sequence to SOC analysts.

1.2 Core Objectives

- Ingest high-velocity, semi-structured system and network telemetry logs into a scalable Document Store (MongoDB).
- Transform log events into a dynamic Temporal Graph (Neo4j) maintaining index-free adjacency for instant path traversal.
- Compute continuous Risk Propagation across graph entities based on CVSS vulnerabilities, anomaly scores, and temporal decay.
- Generate Explainable Attack Paths using Cypher queries to highlight compromised vectors leading to critical assets.

1.3 System Scope

The system ingests Windows Sysmon event logs, Linux Auditd records, authentication logs, and network flow data. MongoDB handles append-heavy raw telemetry and auditing, while Neo4j maintains active topological graph states for real-time risk scoring and path queries.

2. Selection and Justification of NoSQL Database (3 Marks)

2.1 Why Polyglot NoSQL (MongoDB + Neo4j)?

No single database model optimal for both high-velocity append-heavy log streams and complex graph traversals. Hence, a Polyglot NoSQL approach is adopted:

- **MongoDB (Document Store):** Provides high write throughput for unstructured/semi-structured BSON telemetry logs. It allows schema-less ingestion of varying log formats (Sysmon Process Creation, Auth Events, NetFlow) without schema lock overhead.
- **Neo4j (Graph Database):** Utilizes index-free adjacency where graph nodes reference connected neighbors directly via pointers. This guarantees constant-time traversal O(1) per hop regardless of total database size, ideal for multi-stage lateral attack path discovery.

2.2 NoSQL Database Comparison Matrix

Feature / Criteria	Relational DB (SQL)	MongoDB (Document)	Neo4j (Graph)	Our Polyglot Choice
Log Ingestion Speed	Low (Schema overhead)	**Very High (Append BSON)**	Moderate	**MongoDB** for raw log ingestion
Schema Flexibility	Rigid (Migrations)	**Dynamic Schema-less**	Flexible Node/Edge attributes	**MongoDB** for varied log formats
Multi-Hop Traversal	Slow (O(k^d) JOINs)	Slow (Application-side)	**Optimal (Index-free O(1))**	**Neo4j** for lateral movement paths
Risk Propagation	Complex Recursive SQL	Inefficient Map-Reduce	**Native Graph Algorithms**	**Neo4j** Cypher graph traversals

3. Data Model Design (5 Marks)

3.1 ER-to-NoSQL Mapping Strategy

Relational Concept	MongoDB Collection / Field	Neo4j Graph Element
Host (Asset)	db.asset_inventory (Document)	(:Host {host_id, ip, criticality, risk_score})
User Account	db.users (Document)	(:User {username, domain, is_privileged})
Process Execution	db.raw_telemetry.process (Embedded)	(:Process {pid, name, cmd_hash, risk_score})
Vulnerability	db.asset_inventory.vulnerabilities	(:Vulnerability {cve_id, cvss_score})
Auth / Network Event	db.raw_telemetry (Event Doc)	[:AUTHENTICATED_TO], [:CONNECTED_TO], [:SPAWNED]

3.2 MongoDB Collection Schema (Document Design)

Sample BSON schema for raw_telemetry collection storing high-velocity process execution logs:

```
{
  "_id": ObjectId("66c7f1a8e4b0a123456789ab"),
  "timestamp": ISODate("2026-08-22T14:32:10Z"),
  "event_type": "ProcessCreation",
  "host": { "host_id": "HOST-FIN-01", "ip": "10.0.4.15" },
  "user": { "username": "jsmith", "domain": "CORP" },
  "process": {
    "pid": 4120, "name": "powershell.exe",
    "cmd": "powershell -ep bypass -c Invoke-Mimikatz"
  },
  "network": { "dst_ip": "194.26.29.112", "dst_port": 443 }
}
```

3.3 Neo4j Graph Schema (Nodes & Temporal Relationships)

Nodes: (:Host), (:User), (:Process), (:IPAddress), (:Vulnerability)
Temporal Edges: [:AUTHENTICATED_TO {timestamp}], [:SPAWNED {timestamp}], [:LATERAL_MOVE_TO {timestamp}]

3.4 Risk Propagation Model

Dynamic risk score for entity node v at time t is computed as:

$$R(v, t) = R_{base}(v) + \Sigma [R(u, t - \Delta t) \times w(u \rightarrow v) \times e^{(-\lambda (t - t_{edge}))}]$$

Where R_{base} is vulnerability severity (CVSS), $w(u \rightarrow v)$ is edge action weight, and $e^{(-\lambda t)}$ represents temporal risk decay.

4. Expected Deliverables & Sample Dataset Design

4.1 Requirement Specifications

- **FR-01 (Log Ingestion):** Ingest raw security logs into MongoDB raw_telemetry collection.
- **FR-02 (Graph Streaming):** Parse MongoDB log documents into Neo4j nodes and temporal edges (< 2s latency).
- **FR-03 (Temporal Constraint Filter):** Enforce chronological validity ($t1 \leq t2 \leq ... \leq tn$) on attack paths.
- **FR-04 (Explainable Path Query):** Return node-by-node Cypher attack path visualization for SOC analysts.
- **NFR-01 (Performance):** Support ≥ 2,000 logs/sec in MongoDB and < 150ms 6-hop Cypher queries in Neo4j.

4.2 Executable Cypher Query for Explainable Attack Paths

```
// Cypher Query: Discover Chronological Explainable Attack Path
MATCH path = (u:User)-[r1:AUTHENTICATED_TO]->(h1:Host)-[r2:HOSTS_PROCESS]->(p:Process)-[r3:LATERAL_M
OVE_TO]->(target:Host {criticality: 'CRITICAL'})
WHERE r1.timestamp <= r2.timestamp
  AND r2.timestamp <= r3.timestamp
  AND r3.timestamp <= r1.timestamp + duration('PT2H')
RETURN
  u.username AS Attacker, h1.host_id AS EntryHost, p.cmd AS MaliciousCommand,
  target.host_id AS TargetAsset, nodes(path) AS AttackPathNodes,
  reduce(totalRisk = 0.0, n IN nodes(path) | totalRisk + n.risk_score) AS CumulativePathRisk
ORDER BY CumulativePathRisk DESC LIMIT 5;
```

Deliverable Status: Review 1 Report complete and ready for evaluation.